

NexaPay QA automation suite

Risk-based API and end-to-end tests for the NexaPay operations dashboard, written for the QA Automation Senior assessment.

Samuel, 15 August 2026.

Getting it running

```
npm ci # about 20 seconds
cp .env.example .env # then fill in the URL and the three passwords
npm run test:api # about 20 seconds, no browser needed
npm run report # opens the HTML report
```

For the browser tests, install Chromium first:

```
npx playwright install chromium
npm run evidence # screenshots of every screen, into evidence/screenshots/
npm test # everything: api, consistency, e2e
```

If you cannot reach the demo host from wherever you are running this, there is a replay mode that needs no network at all:

```
TEST_MODE=recorded npm run test:api
```

Why Playwright and TypeScript

The assessment asks for a justification, so here is the real reasoning rather than a preference.

One tool covers both levels. The required deliverables span an end-to-end journey and a set of API tests, and one scenario has to compare an API payload against what the UI renders inside a single test. Playwright's `APIRequestContext` and its browser context live in the same process, so that comparison is about ten lines. Splitting API and UI across two runners would have turned the test I most wanted to be easy into the one most likely to get dropped.

It also matches what the team already has. Annexe H shows the existing preprod suite running on Jest and Playwright. Handing over something they can already read is worth more than a marginally better tool nobody knows. I dropped Jest because Playwright's own runner covers what it was doing.

TypeScript earns its place on a payments domain. `Transaction`, `Contact` and `TransferResponse` are compile-time contracts, so a shape change breaks the build instead of producing a confusing failure at runtime. `npm run typecheck` runs before the tests in CI for that reason.

What I left out: no Cucumber, because nobody non-technical is writing these, so the feature-file layer would be indirection with no audience. No Allure, because the built-in HTML reporter plus JUnit XML already covers what was asked and it is one less dependency to keep alive. Ajv is the only real addition, since contract validation was explicitly required and hand-written shape checks rot quickly.

Layout

<code>playwright.config.ts</code>	three projects: <code>api</code> , <code>consistency</code> , <code>e2e</code>
<code>src/</code>	
<code>clients/api-client.ts</code>	timed HTTP wrapper, attaches every call to the report
<code>pages/</code>	page objects, locators only, no assertions
<code>schemas/index.ts</code>	Ajv, published contract vs observed shapes
<code>fixtures/recorded/</code>	verbatim captures with a provenance note
<code>support/</code>	
<code>env.ts</code>	config, throws loudly if something is missing
<code>guards.ts</code>	skip-instead-of-guess helpers
<code>money.ts</code>	integer cent arithmetic

redact.ts	strips PIN, password and tokens from evidence
tests/	
api/	authorisation, idempotency, validation, contract, consistency, observability
consistency/	API against UI
e2e/	critical journey, RBAC, visual evidence
tools/mock-server.mjs	replay server for TEST_MODE=recorded
evidence/	reports, screenshots, traces

Decisions worth explaining

The assertion messages are half the deliverable. Every expect carries a message naming the rule, the observed value and what it costs. A failing test here can be pasted straight into a ticket. `expect(res.status).toBe(403)` is correct and useless.

Everything monetary is compared in integer cents. `src/support/money.ts` converts before comparing. Adding 89.99 and -127.85 as floats gives you order-dependent results, and a payments suite must not go red on float drift or green because a tolerance swallowed a real cent.

Config fails loudly. `env.ts` throws on a missing variable instead of defaulting. A CI job with no `NEXAPAY_BASE_URL` should fail immediately, not pass quietly against localhost.

No secrets in the repo. `.env` is git-ignored, `.env.example` ships with empty values, CI reads from the encrypted store, and gitleaks runs on every push. `redact.ts` scrubs pin, password, token and Authorization out of every report attachment, so the reports are safe to share.

Tests cannot leak into each other. The `api` fixture builds a fresh request context per test. Browser tests run single-worker, because the demo has one shared mutable dataset and parallel writes would produce failures nobody can reproduce. That is a deliberate trade of speed for determinism.

Assertions are about invariants, not values. Things like “the status filters partition the ledger exactly” and “every recipientId resolves to a contact”, rather than hard-coded amounts. The seeded data already differs from what Annex F.6 documents, so a suite pinned to specific figures would have been broken on arrival.

Execution modes

`live` is the default and runs against the real host. `recorded` replays captured payloads from a local server, for when there is no route to the demo.

The replay server follows one rule: it only replays what was actually observed. Anything else returns 501 `not_recorded` and the test skips.

That cuts both ways deliberately. It reproduces the defects, so `/users` answers 200 without an Authorization header here exactly as it does on the demo. But it invents nothing: `POST /transfers` validation was never observed working, so the server does not guess at it. A permissive mock would make ten “must be rejected” tests pass and manufacture false confidence. A strict one would make them fail and manufacture defects nobody can reproduce. Both are fiction.

So a recorded run reports 24 skips. That number is the honest measure of what the environment let me verify.

Proof of execution

After any run:

- `evidence/html-report/index.html`, browsable, with every request and response
- `evidence/screenshots/`, full-page screenshots of each screen
- `evidence/junit.xml`, for CI
- `evidence/results.json`

- evidence/traces/, Playwright traces

`npm run evidence` on its own produces just the screenshot pack, which is the quickest way to see that the suite really drives the application.

What this covers

ID	Scenario	Level	Priority
TS-01	Unauthenticated read of users and contacts	API	P1
TS-02	Tenant scoping on the ledger	API	P1
TS-03	Idempotency on POST /transfers	API	P1
TS-04	Server-side amount, PIN, note and date validation	API	P1
TS-05	Sign in, transfer, confirmation	E2E	P1
TS-06	Schema and contract validation	API	P2
TS-07	Cross-endpoint referential and arithmetic consistency	API	P2
TS-08	API against UI reconciliation	E2E	P1
TS-09	Filter enum validation	API	P2
TS-10	RBAC in the UI, health probes, correlation id	Mixed	P2/P3

Design rationale for each is in `docs/03-test-design.md`.

Known limits

The demo backend is a `json-server` fixture rather than NexaPay's real service. `GET /db` returns the whole database and `POST {"hello": "world"}` returns 201, so no server-side rule can be validated here. The 14 transfer tests are written and reviewed but have never run against real behaviour. Details in `docs/08-live-verification.md`.

Note that Annexe H's existing suite uses `data-testid=row`, which does not exist in the deployed application. The real hook is `tx-row`. That is why TC-04 times out.

Performance checks here are guard-rails, not load tests. TS-07e measures single-user p95 over ten samples from the client side. The Annexe D targets need a proper load tool against a production-sized environment.

The settlement webhook has no coverage, because it needs a signing secret.